# APM52009 – Machine Learning for Scientific Computing and Numerical Analysis

Week 7: Operator Learning

**Victorita Dolean and Hadrien Montanelli**

**Abstract**

This chapter introduces operator learning as a data-driven framework for approximating solution operators of PDEs, with a particular focus on neural operators. Motivated by the limitations of classical reduced-order modeling when solution manifolds are not well approximated by low-dimensional linear spaces, we present neural operators as discretization-invariant mappings between function spaces. A unified lifting–kernel–projection formulation is introduced to organize and compare major neural operator paradigms, including DeepONets, Fourier Neural Operators, and Graph Neural Operators. Through this taxonomy, we highlight the role of inductive biases—such as low-rank structure, spectral representations, and geometric localities—in shaping approximation efficiency and generalization. The chapter connects neural operator learning to reduced-order modeling, discusses theoretical universality results, and illustrates how different architectural choices reflect distinct assumptions on the underlying solution operator.

## Contents

**1 Why operator learning?** **2**

**2 The supervised operator learning problem** **3**

**3 A taxonomy of neural operators** **4**

3.1 Encoder–decoder architectures . . . . . 4

3.2 General neural operators . . . . . 5

**4 Examples of neural operators** **8**

4.1 DeepONets . . . . . 8

4.2 Fourier Neural Operators . . . . . 11

**5 Exercises** **14**

**6 Suggested reading** **15**

# 1 Why operator learning?

Many problems in scientific computing involve the repeated evaluation of a *solution operator*

$$\mathcal{N} : \mathcal{P} \to V, \qquad \mu \mapsto \mathcal{N}(\mu) = u(\cdot;\mu),$$

mapping PDE coefficients, source terms, boundary data, or initial conditions $\mu$ to the corresponding solution of a PDE $u(\cdot;\mu)$. Such many-query settings arise naturally in design and optimization, uncertainty quantification, inverse problems, and real-time control. Reduced-order modeling (ROM) addresses this challenge by approximating the *solution manifold*

$$\mathcal{M} = \{u(\cdot;\mu) : \mu \in \mathcal{P}\} \subset V$$

with a low-dimensional linear subspace $V_N \subset V$, constructed offline from solution snapshots. Online evaluation then amounts to projecting the governing equations onto $V_N$ and solving a reduced system. When the Kolmogorov $N$-width of $\mathcal{M}$ decays rapidly, this strategy yields accurate and efficient surrogates with strong theoretical guarantees; in particular, the corresponding singular values from the POD/SVD of the snapshot matrix also decay rapidly.

**When classical model reduction is insufficient** In many practically relevant regimes, the solution manifold is poorly approximated by low-dimensional linear spaces. This includes, for instance, wave propagation at high frequency, transport-dominated dynamics, and problems with moving fronts or sharp layers. In such cases, the dimension required for an accurate reduced space becomes prohibitively large, undermining the efficiency of projection-based ROMs even when advanced basis construction or localization techniques are used.

**Shift of perspective: learning the operator** Operator learning proposes a complementary viewpoint: rather than approximating individual states in a reduced space, one seeks to directly approximate the operator $\mathcal{N}$ itself as a map between function spaces. The goal is to construct a surrogate operator

$$\mathcal{N}_\theta \approx \mathcal{N}$$

from input–output samples, such that $\mathcal{N}_\theta(\mu)$ can be evaluated rapidly for new inputs $\mu$, without solving a reduced or full PDE. From this perspective, operator learning replaces several core ingredients of classical ROMs by data-driven analogues, as summarized in Table 1.

Table 1: Conceptual dictionary between classical reduced-order modeling and neural operator learning.

| **Reduced-order modeling** | **Neural operator learning** |
|---|---|
| Approximate the solution manifold $\mathcal{M} \subset V$ | Learn the solution operator $\mathcal{N} : \mathcal{P} \to V$ |
| Offline snapshot generation | Training dataset generation $\{(\mu_i, u_i)\}_{i=1}^n$ |
| Reduced basis $\{\psi_k\}_{k=1}^N \subset V$ | Latent feature representation $v_\ell : D \to \mathbb{R}^{d_\ell}$ |
| Projection onto $V_N = \mathrm{span}\{\psi_k\}$ | Nonlinear lifting $\mathcal{R} : \mathcal{P} \to L^2(D;\mathbb{R}^{d_0})$ |
| Coefficient map $\mu \mapsto (c_1, \ldots, c_N)$ | Latent evolution via operator layers $\mathcal{T}_1, \ldots, \mathcal{T}_L$ |
| Linear approximation in $V_N$ | Nonlinear approximation in latent space |
| Online stage: reduced solve | Inference: forward pass |
| Intrusive PDE access | Non-intrusive, data-driven |
| A priori error analysis | Data-dependent generalization |
| Efficient under fast $N$-width decay | Efficient under architecture-aligned operator structure |

In particular, linear reduced spaces are replaced by nonlinear latent representations, intrusive projection steps by non-intrusive training, and state approximation by operator approximation.

Neural operators form a central class of operator learning methods designed to act directly on functions rather than finite-dimensional vectors. They combine discretization-invariant formulations with architectures inspired by integral, spectral, or kernel operators, enabling generalization across input resolutions.

## 2 The supervised operator learning problem

Operator learning seeks to approximate a nonlinear map between function spaces from finite data. We cast it as supervised learning, separating training and generalization, and derive the corresponding error decomposition.

**Setup** Let $d \geq 1$ and $D \subset \mathbb{R}^d$ be a compact domain. We consider Banach spaces $U$ and $V$ of real-valued functions on $D$, representing input and output quantities, respectively. Let $\mathcal{P} \subset U$ be a compact set of admissible inputs and let

$$\mathcal{N} : U \to V$$

be a nonlinear operator mapping an input function $\mu$ to an output function $u = \mathcal{N}(\mu)$. Assume $\mathcal{N}$ is continuous on $\mathcal{P}$ and belongs to $L^2_\sigma(\mathcal{P}, V)$ for some probability measure $\sigma$. In practice, it is observed only through finitely many samples

$$\{(\mu_i, u_i)\}_{i=1}^n, \qquad \mu_i \in \mathcal{P}, \qquad u_i = \mathcal{N}(\mu_i) \in V.$$

We approximate $\mathcal{N}$ by a parametric model $\mathcal{N}_\theta$ with learnable parameters $\boldsymbol{\theta} \in \Theta \subset \mathbb{R}^p$. We denote by

$$u_\theta(\cdot; \mu) = \mathcal{N}_\theta(\mu)$$

the predicted output function corresponding to the input function $\mu$. At this level of abstraction, no assumption is made on the internal structure of $\mathcal{N}_\theta$. Section 3 will show how different neural operator architectures correspond to specific choices of representation for this mapping.

**Errors** The *generalization error* measures how well $\mathcal{N}_\theta$ approximates $\mathcal{N}$ over the whole parameter domain:

$$\mathcal{E}_G(\boldsymbol{\theta}) = \|\mathcal{N}_\theta - \mathcal{N}\|^2_{L^2_\sigma(\mathcal{P},V)} = \int_{\mathcal{P}} \|\mathcal{N}_\theta(\mu) - \mathcal{N}(\mu)\|^2_V \, d\sigma(\mu).$$

Defining this integral rigorously is subtle: $\mathcal{N}_\theta(\mu)$ is Banach-valued and $\mathcal{P}$ may be infinite-dimensional. Formally, $\sigma$ is a Borel measure and the integral is a Bochner integral, a generalization of the Lebesgue integral to Banach-valued functions. The best approximation corresponds to parameters

$$\boldsymbol{\theta}^*_G = \arg\min_{\boldsymbol{\theta} \in \Theta} \mathcal{E}_G(\boldsymbol{\theta}).$$

Since $\mathcal{N}$ is unknown on $\mathcal{P}$, we replace the integral by the *training error* at the samples:

$$\mathcal{E}_T(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \|\mathcal{N}_\theta(\mu_i) - \mathcal{N}(\mu_i)\|^2_V, \qquad \boldsymbol{\theta}^*_T = \arg\min_{\boldsymbol{\theta} \in \Theta} \mathcal{E}_T(\boldsymbol{\theta}).$$

An optimization algorithm (often gradient-based) produces an approximate minimizer $\boldsymbol{\theta}$.

**Remark 2.1.** For operators that are uniformly bounded on $\mathcal{P}$, one may also consider a stronger notion of error based on the $L^\infty$ norm, which emphasizes worst-case behavior. The corresponding generalization and training errors are defined as

$$\mathcal{E}^\infty_G(\boldsymbol{\theta}) = \sup_{\mu \in \mathcal{P}} \|\mathcal{N}_\theta(\mu) - \mathcal{N}(\mu)\|_V, \qquad \mathcal{E}^\infty_T(\boldsymbol{\theta}) = \max_{1 \leq i \leq n} \|\mathcal{N}_\theta(\mu_i) - \mathcal{N}(\mu_i)\|_V.$$

Since $\|\cdot\|_{L^2_\sigma(\mathcal{P},V)} \leq \|\cdot\|_{L^\infty(\mathcal{P},V)}$, uniform control implies control in mean square.

As in Chapter 1, we have the following error decomposition theorem.

**Theorem 2.2** (Error decomposition)**.** *For any parameter $\boldsymbol{\theta} \in \Theta$,*

$$\mathcal{E}_G(\boldsymbol{\theta}) \leq \underbrace{\mathcal{E}_G(\boldsymbol{\theta}^*_G)}_{\text{approximation error}} + \underbrace{2 \sup_{\boldsymbol{\theta}' \in \Theta} |\mathcal{E}_G(\boldsymbol{\theta}') - \mathcal{E}_T(\boldsymbol{\theta}')|}_{\text{generalization gap}} + \underbrace{\mathcal{E}_T(\boldsymbol{\theta}) - \mathcal{E}_T(\boldsymbol{\theta}^*_T)}_{\text{optimization error}}.$$

This decomposition highlights the role of architectural design: the hypothesis class determines the approximation error, its inductive bias affects the generalization gap, and the training algorithm controls the optimization error. These three aspects will reappear repeatedly when comparing neural operator paradigms in Section 3.

**Discrete vs. continuous** In the previous chapter, learning problems were formulated in finite-dimensional spaces, typically with vectors $\boldsymbol{\mu} \in \mathcal{P} \subset \mathbb{R}^{d_P}$ and discrete solutions in $\mathbb{R}^{N_h}$. Operator learning can be seen as the continuous analogue of this setting, where both inputs and outputs are functions.

| | Discrete error (training) | Continuous error (generalization) |
|---|---|---|
| vectors in $\mathcal{P}$ and $\mathbb{R}^{N_h} \approx V_h$ | $\mathcal{E}_T(B)$ | $\mathcal{E}_G(B)$ |
| functions in $\mathcal{P}$ and $V$ | $\mathcal{E}_T(\boldsymbol{\theta})$ | $\mathcal{E}_G(\boldsymbol{\theta})$ |

For vectors $\boldsymbol{\mu} \in \mathcal{P}$ and $U_h(\boldsymbol{\mu}) \in \mathbb{R}^{N_h} \approx V_h$ we minimized, for matrices such that $B^\top B = I_N$,

$$\mathcal{E}_T(B) = \sum_{j=1}^{n} \|U_h(\boldsymbol{\mu}_j) - BB^\top U_h(\boldsymbol{\mu}_j)\|_2^2, \qquad \mathcal{E}_G(B) = \int_{\mathcal{P}} \|U_h(\boldsymbol{\mu}) - BB^\top U_h(\boldsymbol{\mu})\|_2^2 d\boldsymbol{\mu}.$$

**Functional analysis** The infinite-dimensional nature of the spaces $U$ and $V$ raises fundamental questions concerning approximation and generalization. We recall several classical notions from functional analysis.

**Definition 2.3** (Basic functional analysis)**.** Let $U$ and $V$ be Banach spaces.

1. *Separability.* The space $U$ is separable if it contains a countable dense subset.
2. *Compact subsets.* A subset $\mathcal{P} \subset U$ is compact if every sequence $(\mu_n)_{n\in\mathbb{N}} \subset \mathcal{P}$ admits a subsequence $(\mu_{n_k})_{k\in\mathbb{N}}$ converging to some $\mu \in \mathcal{P}$.
3. *Compact operators.* A bounded linear operator $\mathcal{L} : U \to V$ is compact if, for every bounded set $B \subset U$, the image $\mathcal{L}(B)$ is relatively compact in $V$, i.e., $\overline{\mathcal{L}(B)}$ is compact in $V$.
4. *Finite-rank operators.* A bounded linear operator $\mathcal{L} : U \to V$ is said to be of finite rank if its range $\mathcal{L}(U)$ is a finite-dimensional subspace of $V$.
5. *Approximation property.* The space $U$ is said to have the approximation property if, for every compact operator $\mathcal{K} : U \to U$ and every $\varepsilon > 0$, there exists a finite-rank bounded linear operator $\mathcal{F} : U \to U$ such that $\|\mathcal{K} - \mathcal{F}\|_{\mathcal{L}(U)} < \varepsilon$,

# 3 A taxonomy of neural operators

## 3.1 Encoder–decoder architectures

We start from a broad class of operator-learning architectures that factorize a map $\mathcal{N} : \mathcal{P} \to V$ through a finite-dimensional latent space. These architectures introduce an *encoder*

$$\mathcal{F}_P : \mathcal{P} \to \mathbb{R}^{d_P}$$

that maps the input function to a finite-dimensional representation, and a *decoder*

$$\mathcal{G}_V : \mathbb{R}^{d_V} \to V$$

that maps latent vectors to the output function. Between the encoder and decoder, a nonlinear map

$$f_\theta : \mathbb{R}^{d_P} \to \mathbb{R}^{d_V}$$

is trained to capture the operator's action. The resulting approximation has the factorized form

$$\mathcal{N}_\theta = \mathcal{G}_V \circ f_\theta \circ \mathcal{F}_P.$$

This simple encoder–decoder idea was the starting point for neural operator research and led directly to the development of DeepONets [12].

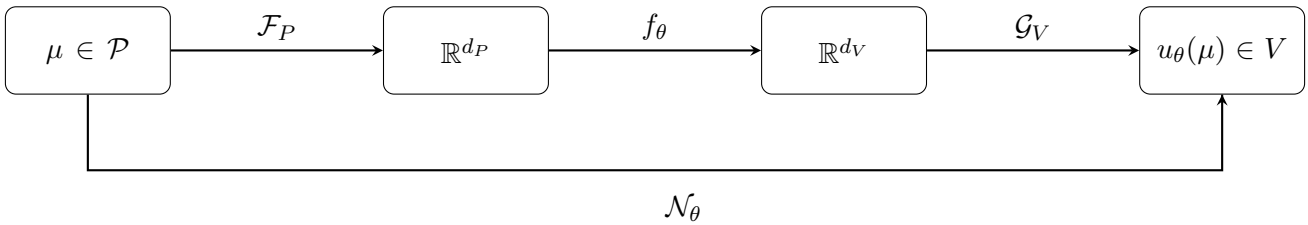


Figure 1: General encoder–decoder operator architecture.

**Definition 3.1** (Encoder–decoder neural operator)**.** An encoder–decoder neural operator is a map

$$\mathcal{N}_\theta : \mathcal{P} \to V, \qquad \mathcal{N}_\theta = \mathcal{G}_V \circ f_\theta \circ \mathcal{F}_P$$

factorized through a finite-dimensional latent space. It consists of

- an encoder $\mathcal{F}_P : \mathcal{P} \to \mathbb{R}^{d_P}$ mapping the input to a latent vector space;
- a nonlinear map $f_\theta : \mathbb{R}^{d_P} \to \mathbb{R}^{d_V}$ acting on the latent representation;
- a decoder $\mathcal{G}_V : \mathbb{R}^{d_V} \to V$ mapping back to the output function $u_\theta$.

This family of encoder–decoder architectures inherently imposes a finite-dimensional bottleneck. Motivated by the desire to avoid this restriction, more general neural operator architectures were subsequently introduced.

## 3.2 General neural operators

Neural operators aim to approximate nonlinear maps between function spaces in a way that is independent of a particular discretization. Rather than producing a single reduced representation of the solution, they learn how functions are mapped to functions. A general neural operator is a parametrized map of the form

$$\mathcal{N}_\theta : \mathcal{P} \to V, \qquad \mathcal{N}_\theta = \mathcal{Q} \circ \mathcal{T}_L \circ \cdots \circ \mathcal{T}_1 \circ \mathcal{R},$$

where $\mathcal{R}$ is a lifting operator, the $\mathcal{T}_\ell$'s are kernel-based operator layers, and $\mathcal{Q}$ is a projection operator. This general view encompasses Fourier neural operators, graph neural operators, and other kernel-based methods, in contrast with the finite-dimensional encoder–decoder approach.

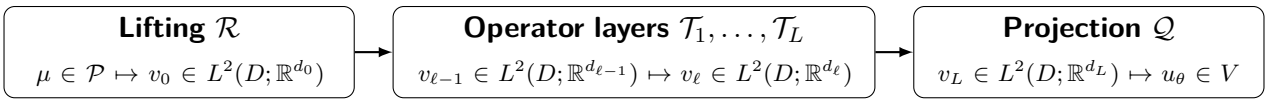


Figure 2: Schematic view of a neural operator: lifting $\to$ kernel-based operator layers $\to$ projection.

**Definition 3.2** (General neural operators)**.** A neural operator is a parametrized mapping

$$\mathcal{N}_\theta : U \to V, \qquad \mathcal{N}_\theta = \mathcal{Q} \circ \mathcal{T}_L \circ \cdots \circ \mathcal{T}_1 \circ \mathcal{R},$$

where:

- $\mathcal{R} : \mathcal{P} \to L^2(D;\mathbb{R}^{d_0})$ is a *lifting operator* mapping the input function into a latent feature space;
- each $\mathcal{T}_\ell : L^2(D;\mathbb{R}^{d_{\ell-1}}) \to L^2(D;\mathbb{R}^{d_\ell})$ is an operator layer of the form

$$v_\ell(\boldsymbol{x}) = \mathcal{T}_\ell v_{\ell-1}(\boldsymbol{x}) = \sigma\left(W^{(\ell-1)} v_{\ell-1}(\boldsymbol{x}) + \int_D K_\theta^{(\ell-1)}(\boldsymbol{x},\boldsymbol{y})\, v_{\ell-1}(\boldsymbol{y})\, d\boldsymbol{y}\right), \quad 1 \le \ell \le L; \tag{3.1}$$

- $\mathcal{Q} : L^2(D;\mathbb{R}^{d_L}) \to V$ is a *projection operator* mapping latent representation to output space.

Here $K_\theta^{(\ell-1)}$ is a learnable kernel, $W^{(\ell-1)}$ a pointwise linear operator, and $\sigma$ a nonlinear activation.

**Remark 3.3** (Learned kernels and latent representations)**.** In Eq. (3.1), the kernel $K_\theta(\boldsymbol{x},\boldsymbol{y})$ is a *learned integral kernel* acting on latent feature fields rather than directly on the input or output functions.

The main differences between neural operator paradigms arise from

- how the kernel $K_\theta$ is parameterized or approximated;
- how the integral interaction is discretized or replaced;
- the structural assumptions imposed by geometry or physics.

We now describe several prominent neural operator families as special cases of this general template.

**Fourier Neural Operators (FNOs)** In FNOs, the integral operator in Eq. (3.1) is parameterized in the *spectral domain*. Assuming $D$ is a regular, periodic domain, the latent field $v_k$ is expanded in a truncated Fourier basis, and the kernel action is realized as a multiplication in frequency space:

$$\int_D K_\theta(\boldsymbol{x}, \boldsymbol{y})\, v_k(\boldsymbol{y})\, d\boldsymbol{y} \quad \longleftrightarrow \quad \mathcal{F}^{-1}\big(R_\theta(\boldsymbol{k})\, \mathcal{F} v_k(\boldsymbol{k})\big),$$

where $\mathcal{F}$ denotes the Fourier transform and $R_\theta(\boldsymbol{k})$ is a learnable matrix-valued multiplier acting on a finite set of low-frequency modes. This corresponds to a translation-invariant kernel $K_\theta(\boldsymbol{x}, \boldsymbol{y}) = K_\theta(\boldsymbol{x} - \boldsymbol{y})$, implicitly represented in Fourier space. FNOs therefore fit naturally into the general framework with a *global, low-rank spectral approximation* of the kernel operator, suited for problems with smooth solutions on structured domains.

**Graph Neural Operators (GNOs)** GNOs arise when the domain $D$ is represented as a graph, typically through a point cloud or mesh. In this setting, the integral interaction in Eq. (3.1) is replaced by a discrete aggregation over neighboring nodes:

$$\int_D K_\theta(\boldsymbol{x}, \boldsymbol{y})\, v(\boldsymbol{y})\, d\boldsymbol{y} \quad \rightsquigarrow \quad \sum_{y \in \mathcal{N}(\boldsymbol{x})} K_\theta(\boldsymbol{x}, \boldsymbol{y})\, v(\boldsymbol{y}),$$

where $\mathcal{N}(\boldsymbol{x})$ denotes the neighborhood of $x$ in the graph. The kernel $K_\theta(\boldsymbol{x}, \boldsymbol{y})$ is often parameterized by a neural network acting on relative positions or edge features. From the present perspective, GNOs correspond to neural operators whose kernel is *geometry-aware* and evaluated on irregular domains, making them particularly suitable for complex geometries where spectral representations are unavailable.

**DeepONets** DeepONets implement a neural operator by separating the dependence on the input function and the spatial variable. Despite their original encoder–decoder structure, DeepONets can be interpreted as neural operators within the general lifting–kernel–projection viewpoint. Within this viewpoint, DeepONets can be interpreted as neural operators with a *finite-rank kernel* of the form

$$K_\theta(\boldsymbol{x}, \boldsymbol{y}) \approx \sum_{r=1}^{R} \phi_r(\boldsymbol{x})\, \psi_r(\boldsymbol{y}),$$

where the basis functions $\phi_r$ and $\psi_r$ are learned implicitly by the trunk and branch networks, respectively. This places DeepONets as a particular low-rank realization of the general operator layer in Eq. (3.1).

| **Operator** | **Kernel representation** | **Interaction structure/typical use cases** |
|---|---|---|
| **FNOs** | Translation-invariant kernel parameterized in Fourier space via learnable spectral multipliers | Global, dense interactions implemented efficiently through FFTs; smooth PDE solutions on regular grids |
| **GNOs** | Geometry-aware kernel defined on graph edges | Local interactions on irregular meshes or point clouds; complex geometries; unstructured discretizations |
| **DeepONets** | Low-rank separable kernel via branch & trunk networks | Global interaction via finite-dimensional latent coefficients; operator regression from sparse sensors |

**Inductive biases of neural operators** Neural operators differ fundamentally in the *inductive biases* they impose on the learned operator. By inductive bias, we mean the set of structural assumptions that an architecture encodes *a priori*, influencing which operators can be learned efficiently from finite data. Within the lifting–kernel–projection framework of Definition 3.2, these biases arise primarily from how the kernel $K_\theta(\boldsymbol{x}, \boldsymbol{y})$ is represented and applied inside the operator layers.

FNOs parameterize the kernel in a truncated Fourier basis, introducing a *spectral bias* toward smooth, global, and translation-invariant interactions with rapidly decaying Fourier coefficients; this makes FNOs particularly effective for elliptic and parabolic PDEs with smooth solutions, but less suitable for operators with strong localization, sharp fronts, or geometry-dependent effects.

GNOs, in contrast, replace the integral interaction with local aggregations over graph neighborhoods, producing a *locality and geometry bias* that naturally encodes the underlying domain or mesh; such architectures excel on irregular domains or manifolds, although long-range interactions must be built implicitly through multiple layers.

DeepONets approximate operators via a finite-rank separable expansion, creating a *low-rank bias* that introduces an explicit latent bottleneck; they are most effective when the solution operator admits a compact, low-dimensional representation, but may require larger latent dimensions when the Kolmogorov $N$-width of the solution manifold decays slowly.

These examples illustrate a central principle of operator learning: *stronger inductive biases often lead to greater data efficiency but reduced generality*, while weaker or more adaptive biases increase expressivity at the cost of higher data and computational requirements.

**Takeaways**

- Neural operators learn maps between function spaces in a discretization-independent manner, generalizing the classical encoder–decoder idea.
- FNOs use a spectral representation of the kernel, introducing a *spectral bias* toward smooth, global, translation-invariant interactions; efficient for PDEs on regular domains, but less suitable for highly localized or geometry-dependent operators.
- GNOs use local, geometry-aware kernel evaluations over graph neighborhoods, producing a *locality and geometry bias*; well-suited for irregular domains or meshes, but long-range interactions require multiple layers.
- DeepONets approximate operators via a finite-rank separable expansion using branch & trunk networks, imposing a *low-rank bias*; they are effective when the solution operator admits a compact, low-dimensional representation, but latent dimension must grow if the solution manifold is complex.
- Stronger inductive biases improve data efficiency but reduce generality; weaker or adaptive biases increase expressiveness at the cost of more data and computation.
- All three paradigms can be interpreted within the general lifting–kernel–projection framework, unifying their treatment and comparison.

# 4 Examples of neural operators

## 4.1 DeepONets

**Definition** DeepONets [12] approximate nonlinear operators through

$$\mathcal{N}_\theta = \mathcal{G}_V \circ f_\theta \circ \mathcal{F}_P.$$

In DeepONets, the input function $\mu \in \mathcal{P}$ is first encoded by sampling at $d_P$ fixed sensors $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{d_P} \in \overline{D}$:

$$\mathcal{F}_P(\mu) = \big(\mu(\boldsymbol{x}_1), \ldots, \mu(\boldsymbol{x}_{d_P})\big)^\top \in \mathbb{R}^{d_P}.$$

The decoder is constructed from a set of basis functions produced by a trainable neural network called the *trunk network*

$$g_\theta : \mathbb{R}^d \to \mathbb{R}^{d_V}, \qquad \mathcal{G}_V(\boldsymbol{y})(\boldsymbol{x}) = \boldsymbol{y}^\top g_\theta(\boldsymbol{x}).$$

The nonlinear map $f_\theta : \mathbb{R}^{d_P} \to \mathbb{R}^{d_V}$ is parametrized through a neural network, called the *branch network*. It computes the coefficients of the learned basis from the encoded input, which yields

$$\mathcal{N}_\theta(\mu)(\boldsymbol{x}) = f_\theta\big(\mathcal{F}_P(\mu)\big)^\top g_\theta(\boldsymbol{x}) = \sum_{k=1}^{d_V} f_{\theta,k}\big(\mu(\boldsymbol{x}_1), \ldots, \mu(\boldsymbol{x}_{d_P})\big)\, g_{\theta,k}(\boldsymbol{x}).$$

The unknowns in the model are the parameters of the branch network $f_\theta$ and the trunk network $g_\theta$.

**Definition 4.1** (DeepONet architecture)**.** A DeepONet approximates an operator $\mathcal{N} : \mathcal{P} \to V$ using a separable representation

$$\mathcal{N}_\theta(\mu)(\boldsymbol{x}) = \sum_{k=1}^{d_V} f_{\theta,k}\big(\mu(\boldsymbol{x}_1), \ldots, \mu(\boldsymbol{x}_{d_P})\big)\, g_{\theta,k}(\boldsymbol{x}),$$

where

- $f_\theta : \mathbb{R}^{d_P} \to \mathbb{R}^{d_V}$ is the branch network producing the coefficient vector from input samples;
- $g_\theta : \mathbb{R}^d \to \mathbb{R}^{d_V}$ is the trunk network producing the basis functions over the physical domain.

The branch and trunk networks $f_\theta$ and $g_\theta$ are the learnable components of the model.

**Relation to reduced-order models** DeepONets can be interpreted as nonlinear reduced-order models: the output is approximated as a learned expansion, mirroring classical ROMs

$$u(\mu)(\boldsymbol{x}) \approx \sum_{k=1}^{N} c_k(\mu)\, \psi_k(\boldsymbol{x}).$$

Here the basis $\{g_{\theta,k}\}$ and the coefficient map $f_\theta$ are learned from data rather than obtained by projection. In contrast to projection-based ROMs, where the coefficients are computed by solving a reduced system derived from the PDE, DeepONets learn this mapping directly from data.

**Training** The trainable parameters of the branch and trunk networks are collected in a vector $\boldsymbol{\theta} \in \mathbb{R}^p$. Given training data $\{(\mu_i, u_i)\}_{i=1}^n$ with $u_i = \mathcal{N}(\mu_i)$, the training error is

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \big\|\mathcal{N}_\theta(\mu_i) - u_i\big\|_V^2.$$

In practice, the norm in $V$ (e.g., a scaled $L^2(D)$-norm) is approximated using quadrature points $\{\boldsymbol{x}_j\}_{j=1}^m$:

$$J(\boldsymbol{\theta}) \approx \frac{1}{n}\frac{1}{m} \sum_{i=1}^{n} \sum_{j=1}^{m} \big|\mathcal{N}_\theta(\mu_i)(\boldsymbol{x}_j) - u_i(\boldsymbol{x}_j)\big|^2.$$

The minimization of $J(\boldsymbol{\theta})$ is performed using gradient-based optimization methods such as SGD or Adam.

**Algorithm 1** DeepONet Training

1: Choose number of sensors $d_P$, basis size $d_V$, sensor locations $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{d_P}$, and architectures for $f_\theta$ and $g_\theta$.

2: Generate training inputs $\mu_i$ (e.g., via random series expansions or Gaussian processes).

3: Compute reference solutions $u_i = \mathcal{N}(\mu_i)$ with standard PDE solvers or PINNs.

4: Choose quadrature points to approximate $\|\cdot\|_V$ (specialized if available & small $d$, else Monte–Carlo).

5: Minimize $J(\boldsymbol{\theta})$ using backpropagation and SGD (or Adam) with, e.g., fixed $h = 10^{-3}$.

**Universality** A central theoretical question is whether architectures of the form $\mathcal{N}_\theta = \mathcal{G}_V \circ f_\theta \circ \mathcal{F}_P$ are expressive enough to approximate general nonlinear operators. Universality of operator networks goes back to 1995 for a slightly simpler architecture [4, Thm. 5].

**Theorem 4.2** (First universality theorem)**.** *Let $\sigma \in C(\mathbb{R})$ be nonpolynomial. Let $D \subset \mathbb{R}^d$ be an open, bounded set with a smooth boundary, and $\mathcal{N} : C(D) \to C(D)$ be continuous. Let $\mathcal{P} \subset C(D)$ be compact. Then for any $\epsilon \in (0,1)$, there exist integers $d_P, d_V, N$, points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{d_P} \in \overline{D}$, and coefficients $c_i^k, \xi_{ij}^k, b_i^k, \zeta_k \in \mathbb{R}$ and $\boldsymbol{w}_k \in \mathbb{R}^d$ such that*

$$\sup_{\mu \in \mathcal{P}} \sup_{\boldsymbol{x} \in D} \left| \mathcal{N}(\mu)(\boldsymbol{x}) - \sum_{k=1}^{d_V} \left[ \sum_{i=1}^{N} c_i^k \sigma \left( \sum_{j=1}^{d_P} \xi_{ij}^k \mu(\boldsymbol{x}_j) + b_i^k \right) \right] \sigma(\boldsymbol{w}_k^\top \boldsymbol{x} + \zeta_k) \right| \leq \epsilon.$$

In our formulation, the linear maps are

$$\mathcal{F}_P(\mu) = (\mu(\boldsymbol{x}_1), \ldots, \mu(\boldsymbol{x}_{d_P}))^\top \in \mathbb{R}^{d_P}, \quad \mathcal{G}_V(\boldsymbol{y})(\boldsymbol{x}) = \sum_{k=1}^{d_V} y_k \sigma(\boldsymbol{\theta}_k^\top \boldsymbol{x} + \zeta_k),$$

while $f_\theta : \mathbb{R}^{d_P} \to \mathbb{R}^{d_V}$ has components

$$f_{\theta,k}(\boldsymbol{x}) = \sum_{i=1}^{N} c_i^k \sigma \left( \sum_{j=1}^{d_P} \xi_{ij}^k x_j + b_i^k \right) \in \mathbb{R}, \quad 1 \leq k \leq d_V.$$

Universality for DeepONets, which guarantees that DeepONet architectures can approximate any continuous operator on a compact set arbitrarily well, was first rigorously established in the original DeepONets paper [12].

**Theorem 4.3** (Universality of DeepONets)**.** *Let $D \subset \mathbb{R}^d$ be an open, bounded set with a smooth boundary, and let $\mathcal{N} : C(D) \to C(D)$ be continuous. Let $\mathcal{P} \subset C(D)$ be compact. Then for any $\epsilon > 0$, there exist integers $d_P, d_V$, sensor points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{d_P} \in \overline{D}$, and networks $f_\theta$ and $g_\theta$ such that*

$$\sup_{\mu \in \mathcal{P}} \|\mathcal{N}(\mu) - f_\theta(\mu(\boldsymbol{x}_1), \ldots, \mu(\boldsymbol{x}_{d_P}))^\top g_\theta\|_{L^\infty(D)} \leq \epsilon.$$

This universality result for DeepONets can be seen as a special case of a more general theoretical framework for encoder–decoder networks, which formalizes (nonlinear) operator approximation in separable Banach spaces; see [8, Lem. 22] and [7, Thm. 4.1].

**Theorem 4.4** (Universality of encoder–decoder networks)**.** *Let $U, V$ be separable Banach spaces with the approximation property. Let $\mathcal{N} : U \to V$ be continuous and $\mathcal{P} \subset U$ compact. Then for any $\epsilon > 0$ there exist integers $d_P, d_V$, bounded linear maps $\mathcal{F}_P : \mathcal{P} \to \mathbb{R}^{d_P}$ and $\mathcal{G}_V : \mathbb{R}^{d_V} \to V$, and a continuous nonlinear map $f_\theta : \mathbb{R}^{d_P} \to \mathbb{R}^{d_V}$ such that*

$$\sup_{\mu \in \mathcal{P}} \|\mathcal{N}(\mu) - \mathcal{N}_\theta(\mu)\|_V \leq \epsilon, \qquad \mathcal{N}_\theta = \mathcal{G}_V \circ f_\theta \circ \mathcal{F}_P.$$

**Quantitative results** Beyond mere universality, it is possible to obtain precise quantitative convergence estimates for DeepONets. In particular, for elliptic PDEs with analytic coefficients and forcing terms, DeepONets exhibit exponential convergence rates with respect to the network size, making them highly efficient for such problems.

Consider the elliptic problem posed on the torus $D = (0, 2\pi)^d$:

$$-\nabla \cdot (\mu \nabla u) = f, \qquad \int_D u = 0,$$

where $f$ and $\mu$ are real analytic, $\mu$ is positive and bounded, and define the set

$$\mathcal{P} = \{\mu : D \to \mathbb{R} \text{ analytic, positive, bounded}\} \subset L^\infty(D).$$

This set $\mathcal{P}$ is compact in $U = L^\infty(D)$. The operator $\mathcal{N} : \mathcal{P} \to V$ maps a coefficient $\mu$ to the corresponding PDE solution $u$, with

$$V = \left\{ u : D \to \mathbb{R} \text{ real analytic}, \int_D u = 0 \right\} \subset H^1(D).$$

**Theorem 4.5** (DeepONets for elliptic PDEs)**.** *For any $\epsilon \in (0, 1)$, there exist integers $d_P, d_V$, sensor locations $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{d_P} \in \overline{D}$, a branch network $f_\theta$, and a trunk network $g_\theta$ such that*

$$\sup_{\mu \in \mathcal{P}} \|\mathcal{N}(\mu) - f_\theta(\mu(\boldsymbol{x}_1), \ldots, \mu(\boldsymbol{x}_{d_P}))^\top g_\theta\|_{H^1(D)} \leq \epsilon,$$

*with*

$$d_P, d_V = \mathcal{O}(\log(1/\epsilon)^d),$$

*and*

$$\begin{aligned} \operatorname{depth}(f_\theta) &= \mathcal{O}(\log(1/\epsilon) \log\log(1/\epsilon)), & \operatorname{size}(f_\theta) &= \mathcal{O}(\log(1/\epsilon)^{3d+2} (\log\log(1/\epsilon))^2), \\ \operatorname{depth}(g_\theta) &= \mathcal{O}(\log(1/\epsilon) \log\log(1/\epsilon)), & \operatorname{size}(g_\theta) &= \mathcal{O}(\log(1/\epsilon)^{d+1}). \end{aligned}$$

**Takeaways**

- DeepONets approximate operators $\mathcal{N} : \mathcal{P} \to V$ via a learned separable expansion

  $$\mathcal{N}_\theta(\mu)(\boldsymbol{x}) = \sum_{k=1}^{d_V} f_{\theta,k}(\mu(\boldsymbol{x}_1), \ldots, \mu(\boldsymbol{x}_{d_P}))\, g_{\theta,k}(\boldsymbol{x}),$$

  where both the coefficient map $f_\theta$ (branch) and the basis $g_\theta$ (trunk) are learned.
- This mirrors classical ROMs, replacing projection-based coefficients and POD bases with learned nonlinear maps.
- Universality ensures expressive power on compact sets.
- Exponential convergence occurs for solution operators of smooth elliptic PDEs.
- The architecture imposes a global low-rank inductive bias, favoring operators with approximately separable structure.
- For strongly nonlocal or multiscale operators, this bias can be restrictive, motivating alternative neural operator designs.

## 4.2 Fourier Neural Operators

FNOs were the first neural operators to adopt a lifting–kernel–projection architecture [10].

**Motivation** Consider the toy elliptic problem

$$\mathcal{L}u = u'' = f \quad \text{on } (-1,1), \qquad u(\pm 1) = 0,$$

with $f \in L^2(-1,1)$. One has

$$\mathcal{L} : H^2(-1,1) \cap H_0^1(-1,1) \to L^2(-1,1), \qquad \mathcal{L}^{-1} : L^2(-1,1) \to H^2(-1,1) \cap H_0^1(-1,1).$$

Suppose we look for a solution $u$ of the form

$$u(x) = (\mathcal{L}^{-1}f)(x) = \int_{-1}^{1} g(x,y)f(y)dy,$$

for some $g : [-1,1]^2 \to \mathbb{R}$. If $g$ satisfies $\mathcal{L}g = g_{xx} = \delta(x-y)$ (in the distributional sense), then (formally)

$$(\mathcal{L}u)(x) = \int_{-1}^{1} g_{xx}(x,y)f(y)dy = \int_{-1}^{1} \delta(x-y)f(y)dy = f(x).$$

This reflects a general, important principle: many PDE solution operators admit an integral representation with a problem-dependent kernel. FNOs encode this structure directly by learning the kernel from data.

**Fourier analysis** Let $D = (0, 2\pi)^d$. Any sufficiently smooth function $f : D \to \mathbb{C}^{d'}$ with components $f_j$, $1 \le j \le d'$, admits a Fourier expansion

$$f_j(\boldsymbol{x}) = \sum_{\boldsymbol{k}\in\mathbb{Z}^d} c_{\boldsymbol{k}}^j \, e^{i\boldsymbol{k}\cdot\boldsymbol{x}}, \quad c_{\boldsymbol{k}}^j = \frac{1}{(2\pi)^d} \int_D f_j(\boldsymbol{x}) \, e^{-i\boldsymbol{k}\cdot\boldsymbol{x}} \, d\boldsymbol{x}.$$

In practice, $f$ is discretized on a uniform grid and approximated by truncated Fourier series (via the FFT).

**Theorem 4.6** (Vector-valued convolution)**.** *Let $f : D \to \mathbb{C}^{d'}$, $g : D \to \mathbb{C}^{d'\times d'}$ with Fourier coefficients*

$$f_j(\boldsymbol{x}) = \sum_{\boldsymbol{k}\in\mathbb{Z}^d} c_{\boldsymbol{k}}^j e^{i\boldsymbol{k}\cdot\boldsymbol{x}}, \qquad g_{jj'}(\boldsymbol{x}) = \sum_{\boldsymbol{k}\in\mathbb{Z}^d} d_{\boldsymbol{k}}^{jj'} e^{i\boldsymbol{k}\cdot\boldsymbol{x}}, \quad 1 \le j, j' \le d'.$$

*Define the vector-valued convolution*

$$h(\boldsymbol{x}) = \int_D g(\boldsymbol{x}-\boldsymbol{y})f(\boldsymbol{y})d\boldsymbol{y} \ \in \mathbb{C}^{d'}.$$

*Then*

$$h_j(\boldsymbol{x}) = \sum_{\boldsymbol{k}\in\mathbb{Z}^d} \sum_{j'=1}^{d'} d_{\boldsymbol{k}}^{jj'} c_{\boldsymbol{k}}^{j'} \, e^{i\boldsymbol{k}\cdot\boldsymbol{x}}, \qquad 1 \le j \le d'.$$

**Proof**

Write the convolution componentwise:

$$h_j(\boldsymbol{x}) = \sum_{j'=1}^{d'} \int_D g_{jj'}(\boldsymbol{x}-\boldsymbol{y}) f_{j'}(\boldsymbol{y}) \, d\boldsymbol{y}.$$

Insert the Fourier expansions:

$$f_{j'}(\boldsymbol{y}) = \sum_{\boldsymbol{m}\in\mathbb{Z}^d} c_{\boldsymbol{m}}^{j'} e^{i\boldsymbol{m}\cdot\boldsymbol{y}}, \qquad g_{jj'}(\boldsymbol{x}-\boldsymbol{y}) = \sum_{\boldsymbol{k}\in\mathbb{Z}^d} d_{\boldsymbol{k}}^{jj'} e^{i\boldsymbol{k}\cdot(\boldsymbol{x}-\boldsymbol{y})}.$$

Thus

$$h_j(\boldsymbol{x}) = \sum_{j'=1}^{d'} \sum_{\boldsymbol{k},\boldsymbol{m}\in\mathbb{Z}^d} d_{\boldsymbol{k}}^{jj'} c_{\boldsymbol{m}}^{j'} e^{i\boldsymbol{k}\cdot\boldsymbol{x}} \int_D e^{i(\boldsymbol{m}-\boldsymbol{k})\cdot\boldsymbol{y}} \, d\boldsymbol{y}.$$

By orthogonality of Fourier modes on $D$, only terms with $\boldsymbol{m} = \boldsymbol{k}$ survive.

**Definition 4.7** (FNO architecture)**.** A Fourier Neural Operator (FNO) is a neural operator of the form

$$\mathcal{N}_\theta = \mathcal{Q}_\theta \circ \mathcal{T}_L \circ \cdots \circ \mathcal{T}_1 \circ \mathcal{R}_\theta,$$

where

- $\mathcal{R}_\theta : \mathcal{P} \to L^2(D; \mathbb{R}^{d_0})$ is a *lifting operator* implemented as a neural network;
- each layer $\mathcal{T}_\ell : L^2(D; \mathbb{R}^{d_{\ell-1}}) \to L^2(D; \mathbb{R}^{d_\ell})$ is defined by

  $$v_\ell(\boldsymbol{x}) = \mathcal{T}_\ell v_{\ell-1}(\boldsymbol{x}) = \sigma\Big(W^{(\ell-1)} v_{\ell-1}(\boldsymbol{x}) + \boldsymbol{b}^{(\ell-1)} + \int_D K_\theta^{(\ell-1)}(\boldsymbol{x}-\boldsymbol{y})\, v_{\ell-1}(\boldsymbol{y})\, d\boldsymbol{y}\Big), \quad 1 \le \ell \le L;$$

  where $K_\theta^{(\ell-1)}$ is the *learnable kernel* of the Fourier convolution (in Fourier space, this corresponds to multiplying each retained mode $\boldsymbol{k}$ by a learnable matrix, typically restricted to $\|\boldsymbol{k}\|_\infty \le k_{\max}$);
- $\mathcal{Q}_\theta : L^2(D; \mathbb{R}^{d_L}) \to V$ is a *projection operator* implemented as a neural network.

**Training** The trainable parameters include the pointwise networks $R_\theta$ and $Q_\theta$, the matrices $W^{(\ell)}$ and biases $\boldsymbol{b}^{(\ell)}$, and the Fourier coefficients of the kernels. Given training pairs $(\mu_i, u_i)$, the loss

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \|\mathcal{N}_\theta(\mu_i) - u_i\|_V^2$$

is approximated using quadrature on a uniform grid. Each convolution is evaluated by:

- applying the FFT to the latent field;
- multiplying retained Fourier modes by learned matrices;
- padding and transforming back via the inverse FFT.

This yields a computational cost that scales quasi-linearly with the number of grid points.

**Algorithm 2** FNO Training

1: Choose architectures for $R_\theta$ and $Q_\theta$, number of layers $L$, latent dimensions $d_\ell$, grid size $m$ for the FFT and maximum retained wavenumber $k_{\max}$.

2: Generate training inputs $\mu_i$ (e.g., via random series expansions or Gaussian processes).

3: Compute reference solutions $u_i = \mathcal{N}(\mu_i)$ with standard PDE solvers or PINNs.

4: Choose quadrature points to approximate $\|\cdot\|_V$ (specialized if available & small $d$, else Monte–Carlo).

5: Minimize $J(\boldsymbol{\theta})$ using backpropagation and SGD (or Adam) with, e.g., fixed $h = 10^{-3}$.

**Universality** A fundamental theoretical question is whether FNOs are expressive enough to approximate general nonlinear operators between function spaces. Universality results answer this question in the affirmative, showing that FNOs an approximate any continuous operator on compact subsets of Sobolev spaces [6, Thm. 5]. We recall that, for $v \in H^s(D)$ with coefficients $c_{\boldsymbol{k}}$,

$$\|v\|_{H^s(D)}^2 = \sum_{\boldsymbol{k} \in \mathbb{Z}^d} (1 + \|\boldsymbol{k}\|_2^2)^s |c_{\boldsymbol{k}}|^2.$$

**Theorem 4.8** (Universality of FNOs)**.** *Let $s, s' \ge 0$ and $\mathcal{N} : H^s(D) \to H^{s'}(D)$ be continuous. Then for any compact $\mathcal{P} \subset H^s(D)$ and $\epsilon \in (0,1)$, there exists a FNO $\mathcal{N}_\theta$ such that*

$$\sup_{\mu \in \mathcal{P}} \|\mathcal{N}(\mu) - \mathcal{N}_\theta(\mu)\|_{H^{s'}(D)} \le \epsilon.$$

**Quantitative estimates** Beyond universality, quantitative convergence rates can be established for specific classes of PDEs. We briefly summarize results for smooth elliptic problems, following [6, Thm. 26].

Let $D = (0, 2\pi)^d$ and consider the elliptic problem

$$-\nabla \cdot (\mu \nabla u) = f, \qquad \int_D u = 0,$$

where $f \in L^2(D)$ satisfies $\int_D f = 0$. The coefficient $\mu$ is assumed to belong to the compact set

$$\mathcal{P} = \left\{ \mu \in H^s(D) : \|\mu\|_{H^s(D)} \leq \tfrac{1}{\alpha},\ \|\mu - 1\|_{L^\infty(D)} \leq 1 - \alpha \right\} \subset L^\infty(D),$$

with $\alpha > 0$ and $s > d/2 + 1$. The associated solution operator $\mathcal{N}$ maps $\mathcal{P}$ into

$$V = \{u \in H^1(D) : \int_D u = 0\} \subset H^1(D).$$

**Theorem 4.9** (FNOs for elliptic PDEs)**.** *For any $\epsilon \in (0, 1)$, there exists a FNO $\mathcal{N}_\theta$ such that*

$$\sup_{\mu \in \mathcal{P}} \|\mathcal{N}(\mu) - \mathcal{N}_\theta(\mu)\|_{H^1(D)} \leq \epsilon,$$

*with network complexity satisfying*

$$\mathrm{depth}(\mathcal{N}_\theta) = \mathcal{O}(\log(1/\epsilon)), \qquad \mathrm{size}(\mathcal{N}_\theta) = \mathcal{O}(\epsilon^{-d} \log(1/\epsilon)).$$

**Takeaways**

- FNOs approximate nonlinear operators $\mathcal{N} : \mathcal{P} \to V$ through a learned integral representation

  $$\mathcal{N}_\theta = Q_\theta \circ a^{(L)} \circ \cdots \circ a^{(1)} \circ R_\theta,$$

  where $R_\theta$ lifts the input to a latent function space, the layers $a^{(\ell)}$ combine pointwise linear maps with learned convolution operators, and $Q_\theta$ projects back to the output space.
- This mirrors classical integral operator formulations of PDE solution maps, with kernels learned from data rather than derived analytically.
- Universality results guarantee approximation of continuous operators on compact subsets of Sobolev spaces.
- For elliptic PDE solution operators, quantitative estimates yield algebraic convergence in network width and logarithmic depth dependence on target accuracy.
- The architecture imposes a translation-invariant integral-operator inductive bias, favoring operators with convolution-type structure.
- FFT-based convolution yields quasi-linear computational complexity in the number of spatial degrees of freedom.

# 5 Exercises

**Exercise 5.1** (Operators vs. discretizations)**.** Let $U = V = L^2(0,1)$ and define the operator

$$(\mathcal{N}\mu)(\boldsymbol{x}) = \int_0^1 k(x,y)\,\mu(y)\,dy,$$

where $k \in L^2\big((0,1)^2\big)$.

1. Show that $\mathcal{N} : L^2(0,1) \to L^2(0,1)$ is a bounded linear operator.
2. Discuss why approximating $\mathcal{N}$ directly is advantageous when the input resolution varies across datasets.

**Exercise 5.2** (Locality in graph-based operators)**.** Let $G = (V,E)$ be a graph and let $\mathcal{N}(\boldsymbol{x})$ denote the neighborhood of a node $\boldsymbol{x} \in V$. Define an operator

$$(\mathcal{T}v)(\boldsymbol{x}) = \sum_{\boldsymbol{y}\in\mathcal{N}(\boldsymbol{x})} K(\boldsymbol{x},\boldsymbol{y})\,v(\boldsymbol{y}).$$

1. Explain how stacking multiple layers of such operators increases the effective range of interactions.
2. Compare this mechanism with global integral operators acting on functions.

**Exercise 5.3** (Translation invariance and convolution kernels)**.** Let $D = \mathbb{T}^d$ be the $d$-dimensional torus and let $K(\boldsymbol{x},\boldsymbol{y}) = \kappa(\boldsymbol{x}-\boldsymbol{y})$ with $\kappa \in L^1(\mathbb{T}^d)$. Define

$$(\mathcal{K}v)(\boldsymbol{x}) = \int_{\mathbb{T}^d} \kappa(\boldsymbol{x}-\boldsymbol{y})\,v(\boldsymbol{y})\,d\boldsymbol{y}.$$

1. Show that $\mathcal{K}$ is translation invariant.
2. Show that $\mathcal{K}$ is diagonalized by the Fourier basis.
3. Explain why truncating high-frequency Fourier modes leads to a low-rank approximation of $\mathcal{K}$.

**Exercise 5.4** (Finite-rank kernels as operator layers)**.** Assume that the kernel $K_\theta(\boldsymbol{x},\boldsymbol{y})$ admits a finite-rank representation

$$K_\theta(\boldsymbol{x},\boldsymbol{y}) = \sum_{r=1}^{R} \phi_r(\boldsymbol{x})\,\psi_r(\boldsymbol{y}),$$

with $\phi_r, \psi_r \in L^2(D)$. Consider the operator layer

$$(\mathcal{T}v)(\boldsymbol{x}) = \sigma\Big(Wv(\boldsymbol{x}) + \int_D K_\theta(\boldsymbol{x},\boldsymbol{y})\,v(\boldsymbol{y})\,d\boldsymbol{y}\Big).$$

1. Interpret this structure as a projection onto a finite-dimensional space followed by a reconstruction.
2. Explain how increasing $R$ affects both the expressiveness and the computational cost of the operator layer.

## 6 Suggested reading

Table 2: *TL;DR table of 15 favorite references.*

| | Articles |
|---|---|
| DeepONet theory & practice | [3, 4, 9, 14] & [12, 13] |
| FNO theory & practice | [6] & [5, 10, 11] |
| Other architectures & surveys | [1, 15] & [2, 7, 8] |

## References


[1] N. Boullé, C. J. Earls, and A. Townsend, *Data-driven discovery of Green's functions with human-understandable deep learning*, Sci. Rep., 12 (2022), p. 4824.

[2] N. Boullé and A. Townsend, *A mathematical guide to operator learning*, arXiv preprint, 2312.14688 (2023).

[3] K. Chen, C. Wang, and H. Yang, *Deep operator learning lessens the curse of dimensionality for PDEs*, Transact. Mach. Learn. Res., 10 (2023).

[4] T. Chen and H. Chen, *Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its applications to dynamical systems*, IEEE Trans. Neural Netw., 6 (1995), pp. 911–917.

[5] V. S. Fanaskov and I. V. Oseledets, *Spectral neural operators*, Dokl. Math., 108 (2024), pp. 226–232.

[6] N. Kovachki, S. Lanthaler, and S. Mishra, *On universal approximation and error bounds for Fourier neural operators*, J. Mach. Learn. Res., 22 (2021), pp. 1–76.

[7] N. Kovachki, S. Lanthaler, and A. Stuart, *Operator learning: Algorithms and applications*, arXiv preprint, 2402.15715 (2024).

[8] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar, *Neural operator: Learning maps between function spaces with applications to PDEs*, J. Mach. Learn. Res., 24 (2023), pp. 1–97.

[9] S. Lanthaler, S. Mishra, and G. E. Karniadakis, *Error estimates for DeepONets: A deep learning framework in infinite dimensions*, IMA Trans. Maths. Appl., 6 (2022), pp. 1–141.

[10] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, *Fourier neural operators for parametric differential equations*, in ICLR Proceedings, 2021.

[11] L. Lingsch, M. Y. Michelis, E. de Bézenac, S. M. Perera, R. K. Katzschmann, and S. Mishra, *Beyond regular grids: Fourier-based neural operators on arbitrary domains*, arXiv preprint, 2305.19663 (2023).

[12] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, *Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators*, Nat. Mach. Intell., 3 (2021), pp. 218–229.

[13] L. Lu, X. Meng, Z. Mao, and G. E. Karniadakis, *DeepXDE: A deep learning library for solving differential equations*, SIAM Rev., 63 (2021), pp. 208–228.

[14] C. Marcati and C. Schwab, *Exponential convergence of deep operator networks for elliptic partial differential equations*, SIAM J. Numer. Anal., 61 (2023), pp. 1513–1545.

[15] R. Molinaro, Y. Yang, B. Engquist, and S. Mishra, *Neural inverse operators for solving PDE inverse problems*, arXiv preprint, 2301.11167 (2023).